

Smart Teacher — File-by-File Deep Dive (Part 3)

Suite de Part 1 et Part 2. Couvre les fichiers restants : voice FSM, toutes les routes REST, scripts utilitaires, observability dashboard, helpers config, et le reste de la pédagogie.

Sommaire

Voice FSM (audio/voice/) 1. [audio/voice/state_machine.py](#) 2. [audio/voice/event_bus.py](#) 3. [audio/voice/command_parser.py](#) 4. [audio/voice/bypass_llm.py](#)

Agentic (compléments) 5. [agentic/orchestrator.py](#) 6. [agentic/observability.py](#) 7. [agentic/profiling.py](#)

Routes REST (toutes) 8. [routes/admin.py](#) 9. [routes/analytics.py](#) 10. [routes/dashboard_services.py](#) 11. [routes/health.py](#) 12. [routes/learning_gain.py](#) 13. [routes/practice.py](#) 14. [routes/rest.py](#) 15. [routes/search.py](#) 16. [routes/voice.py](#) 17. [routes/student_state.py](#) 18. [handlers/rest_routes.py](#)

Pédagogie (compléments) 19. [pedagogy/concept_types.py](#) 20. [pedagogy/course_analyzer.py](#) 21. [pedagogy/slide_sync.py](#) 22. [pedagogy/student_model.py](#) 23. [pedagogy/personalization/learning_style/heuristic.py](#) 24. [pedagogy/personalization/bandit/log_extractor.py](#) 25. [pedagogy/personalization/bandit/offline.py](#) 26. [pedagogy/personalization/bandit/simulator.py](#) 27. [pedagogy/knowledge_graph/persistence.py](#)

Core, AI helpers 28. [core/diagnostics.py](#) 29. [core/domains_config.py](#) 30. [ai/prompt_rules.py](#) 31. [deps.py](#)

RAG helpers 32. [rag/metadata.py](#) 33. [rag/evaluation/](#)

Storage / Observability 34. [storage/transcript_search.py](#) 35. [storage/media_storage.py](#) 36. [observability/dashboard.py](#)

Database 37. [database/crud.py](#) 38. [database/reset_db_fresh.py](#)

Scripts utilitaires 39. [scripts/reset_for_reingest.py](#) 40. [scripts/reset_students_and_cache.py](#) 41. [scripts/bootstrap_bandit.py](#) 42. [scripts/extract_bandit_logs.py](#) 43. [scripts/profile_agentic.py](#) 44. [scripts/run_rag_eval.py](#) 45. [scripts/clear_presentation_cache.py](#) 46. [scripts/e2e_ingestion_test.py](#) 47. [scripts/migrate_phase2_persistence.py](#) 48. [analyze_metrics.py](#)

1. audio/voice/state_machine.py

Rôle : Voice State Machine (VSM) — distinct du DialogState. Gère les états bas-niveau du flux audio (idle / listening / processing / speaking).

```

from enum import Enum
import asyncio
import logging

log = logging.getLogger("SmartTeacher.VoiceFSM")

class VoiceState(str, Enum):
    IDLE = "idle" # WS connecté mais aucun stream actif
    LISTENING = "listening" # VAD actif, capture audio
    PROCESSING = "processing" # STT + LLM en cours
    SPEAKING = "speaking" # TTS en cours
    PAUSED = "paused" # Pause utilisateur

# Valid transitions (separate from DialogState)
_VALID = {
    VoiceState.IDLE: {VoiceState.LISTENING},
    VoiceState.LISTENING: {VoiceState.PROCESSING, VoiceState.IDLE, VoiceState.PAUSED},
    VoiceState.PROCESSING: {VoiceState.SPEAKING, VoiceState.LISTENING, VoiceState.IDLE},
    VoiceState.SPEAKING: {VoiceState.LISTENING, VoiceState.PAUSED, VoiceState.IDLE},
    VoiceState.PAUSED: {VoiceState.LISTENING, VoiceState.IDLE, VoiceState.PROCESSING},
}

class VoiceStateMachine:
    """Single source of truth for voice state per WS session.

    Distinct from DialogState (pedagogy/dialogue.py) :
    - VoiceState : low-level audio FSM (mic + TTS pipeline)
    - DialogState : high-level pedagogical FSM (lesson flow)

    They coexist : student paused (VoiceState=PAUSED) AND in WAITING (DialogState).
    """
    def __init__(self, session_id: str):
        self.session_id = session_id
        self.state = VoiceState.IDLE
        self._lock = asyncio.Lock()
        self._listeners: list = [] # callbacks pour state change

    async def transition(self, target: VoiceState, reason: str = "") -> bool:
        async with self._lock:
            if target not in _VALID.get(self.state, set()):
                log.warning("[%s] VFSM invalid : %s → %s (reason: %s)",
                    self.session_id[:8], self.state.value, target.value, reason)
                return False
            old = self.state
            self.state = target
            log.info("[%s] VFSM : %s → %s (reason: %s)",
                self.session_id[:8], old.value, target.value, reason)
            for cb in self._listeners:
                try:
                    await cb(old, target, reason)
                except Exception as exc:
                    log.debug("VFSM listener error: %s", exc)
            return True

    def subscribe(self, callback):
        self._listeners.append(callback)

class VoiceWatchdog:
    """Watchdog qui surveille les transitions bloquées.

    Si VFSM est en PROCESSING depuis > 60s → force IDLE + log alerte.
    """
    def __init__(self, vfsm: VoiceStateMachine, max_processing_s: float = 60.0):
        self.vfsm = vfsm
        self.max_processing_s = max_processing_s
        self._task: Optional[asyncio.Task] = None

    async def _watchdog_loop(self):

```

```

last_state = self.vfsm.state
last_change = time.time()
while True:
    await asyncio.sleep(5)
    if self.vfsm.state != last_state:
        last_state = self.vfsm.state
        last_change = time.time()
    elif self.vfsm.state == VoiceState.PROCESSING:
        if time.time() - last_change > self.max_processing_s:
            log.warning("[%s] VFSM watchdog : stuck in PROCESSING > %.0fs → force IDLE",
                        self.vfsm.session_id[:8], self.max_processing_s)
            await self.vfsm.transition(VoiceState.IDLE, reason="watchdog_timeout")

def start(self):
    self._task = asyncio.create_task(self._watchdog_loop())

def cancel(self):
    if self._task:
        self._task.cancel()

```

2. audio/voice/event_bus.py

Rôle : pub/sub interne pour découpler les composants audio.

```

from typing import Callable, Awaitable
import asyncio

```

```

class EventBus:
    """Lightweight async event bus for voice-related events.

    Usage:
        bus = EventBus()
        bus.on("speech_start", async_handler)
        await bus.emit("speech_start", {"timestamp": ...})
    """
    def __init__(self):
        self._handlers: dict[str, list[Callable]] = {}

    def on(self, event: str, handler: Callable[[dict], Awaitable]):
        self._handlers.setdefault(event, []).append(handler)

    def off(self, event: str, handler: Callable):
        if event in self._handlers:
            self._handlers[event].remove(handler)

    async def emit(self, event: str, payload: dict = None):
        payload = payload or {}
        for handler in self._handlers.get(event, []):
            try:
                await handler(payload)
            except Exception as exc:
                log.debug("EventBus handler error for %s: %s", event, exc)

```

Events typiques :

- "speech_start" / "speech_end" (VAD)
 - "interrupt_detected"
 - "tts_start" / "tts_end"
 - "audio_chunk_sent"
-

3. audio/voice/command_parser.py

Rôle : parser de commandes vocales explicites (« slide suivante », « ralentis »).

```
COMMANDS_FR = {
    "next_slide": ["slide suivante", "suivant", "page suivante", "prochaine slide"],
    "previous_slide": ["slide précédente", "précédent", "page précédente", "retourne en arrière"],
    "repeat": ["répète", "redis", "répéter"],
    "skip": ["passe", "skip", "saute"],
    "slow_down": ["ralentis", "moins vite", "plus lentement"],
    "speed_up": ["accélère", "plus vite"],
    "pause": ["pause", "stop", "arrête"],
    "resume": ["reprends", "continue", "reprendre"],
    "stop": ["arrête tout", "termine", "quitte"],
}

COMMANDS_EN = {
    "next_slide": ["next slide", "next", "next page"],
    "previous_slide": ["previous slide", "previous", "back"],
    # ...
}

def parse_command(text: str, language: str = "fr") -> Optional[str]:
    """Parse explicit voice command. Returns command name or None."""
    if not text:
        return None
    cmds = COMMANDS_FR if language[:2] == "fr" else COMMANDS_EN
    text_lower = text.strip().lower()
    for cmd_name, phrases in cmds.items():
        for phrase in phrases:
            if phrase in text_lower:
                return cmd_name
    return None
```

Heuristic-based (pas LLM) → instantané, déterministe. Si pas de match → retourne None et le LLM intent classifier prend le relais.

4. audio/voice/bypass_llm.py

Rôle : court-circuiter le LLM pour les commandes navigation simples (pas la peine de payer un appel LLM pour « slide suivante »).

```
async def try_bypass_llm(text: str, ctx, dialogue, send_state) -> bool:
    """If text is a simple voice command, execute directly + return True.
    Otherwise return False → caller continues with full Q&A pipeline.
    """
    cmd = parse_command(text, ctx.language)
    if not cmd:
        return False

    log.info("[%s] ■ Voice command bypass: %s", ctx.session_id[:8], cmd)

    if cmd == "next_slide":
        await dialogue.next_section(session_id)
        return True
    if cmd == "previous_slide":
        await dialogue.prev_section(session_id)
        return True
    if cmd == "pause":
        await dialogue.pause_session(session_id, slide_id=ctx.paused_state.get("slide_id"))
        return True
```

```

if cmd == "resume":
    await dialogue.resume_session(session_id)
    return True
if cmd == "slow_down":
    # Adjust profile speech_rate
    ...
# ... etc
return False

```

Économise ~1-2s de latence par commande de navigation.

5. agentic/orchestrator.py

Rôle : façade haut-niveau qui choisit Q&A vs Teaching selon le contexte.

```

class TutorOrchestrator:
    def __init__(self, brain, rag):
        self.brain = brain
        self.rag = rag
        self.qa_graph = build_qa_graph(brain, rag)
        self.teaching_graph = build_teaching_graph(brain, rag)

    async def answer_question(self, state: TutorState) -> TutorState:
        """Run Q&A graph end-to-end."""
        log.info("[%s] ■ Q&A Graph invoke | turn=%d", state.session_id[:8], state.turn_id)
        timings = {}
        async for update_chunk in self.qa_graph.astream(state, stream_mode="updates"):
            for node_name, node_state in update_chunk.items():
                # Track per-node timing via observability event
                timings[node_name] = round(node_state.get("_latency_s", 0), 1)

        log.info("[%s] ■ Q&A Graph done | intent=%s | timings=%s | %d chars",
                 state.session_id[:8], state.intent.type if state.intent else "?",
                 timings, len(state.answer or ""))
        return state

    async def narrate_slide(self, state: TutorState) -> TutorState:
        """Run Teaching graph end-to-end."""
        log.info("[%s] ■ Teaching Graph invoke", state.session_id[:8])
        async for update_chunk in self.teaching_graph.astream(state, stream_mode="updates"):
            ...
        log.info("[%s] ■ Teaching Graph done | %d chars", state.session_id[:8], len(state.answer))
        return state

```

6. agentic/observability.py

Rôle : observability hooks pour les graphes LangGraph (timing, errors, cost tracking).

```

class GraphObserver:
    """Emits structured events for each graph step.

    Subscribers : CSV logger, ClickHouse, KPI tracker.
    """
    def __init__(self):
        self.event_log: list[dict] = []

    def on_node_start(self, node_name: str, state: TutorState):
        self.event_log.append({
            "type": "node_start",

```

```

        "node":      node_name,
        "session_id": state.session_id,
        "timestamp": time.time(),
    })

def on_node_end(self, node_name: str, state: TutorState, latency_s: float):
    self.event_log.append({
        "type":      "node_end",
        "node":      node_name,
        "session_id": state.session_id,
        "latency_s": latency_s,
        "timestamp": time.time(),
    })
    log.info("agentic.resilience.wrap - resilience event: node=%s kind=ok latency=%.3f",
            node_name, latency_s)

def on_node_error(self, node_name: str, state: TutorState, exc: Exception):
    log.warning("agentic.resilience.wrap - resilience event: node=%s kind=error exc=%s",
              node_name, type(exc).__name__)

```

7. agentic/profiling.py

Rôle : profiler dev pour mesurer où passe le temps dans les graphes.

```

@contextmanager
def profile_node(name: str):
    """Context manager that prints timing breakdown for a node."""
    t0 = time.perf_counter()
    yield
    elapsed = time.perf_counter() - t0
    log.debug("[profile] %s took %.3fs", name, elapsed)

async def profile_qa_run(state: TutorState, brain, rag, n_runs: int = 1) -> dict:
    """Run Q&A graph N times, return aggregated timings."""
    graph = build_qa_graph(brain, rag)
    timings_per_node = defaultdict(list)
    for _ in range(n_runs):
        async for chunk in graph.astream(state):
            for node, st in chunk.items():
                if "_latency_s" in st:
                    timings_per_node[node].append(st["_latency_s"])
    return {
        node: {
            "mean": sum(t) / len(t),
            "p50": sorted(t)[len(t) // 2],
            "p95": sorted(t)[int(len(t) * 0.95)],
            "p99": sorted(t)[int(len(t) * 0.99)],
            "max": max(t),
            "count": len(t),
        }
        for node, t in timings_per_node.items()
    }

```

Utilisé par [scripts/profile_agentic.py](#).

8. routes/admin.py

Rôle : endpoints admin (require role admin/teacher).

```

router = APIRouter(prefix="/admin", dependencies=[Depends(require_admin)])

@router.get("/users")
async def list_users():
    """List all students (admin only)."""
    async with AsyncSessionLocal() as db:
        students = (await db.execute(select(Student).order_by(Student.created_at.desc()))).scalars().all()
        return {"users": [
            {"id": str(s.id), "email": s.email, "first_name": s.first_name,
             "account_level": s.account_level, "is_active": bool(s.is_active),
             "student_level": s.student_level,
             "created_at": s.created_at.isoformat() if s.created_at else None}
            for s in students
        ]}

@router.post("/users/{user_id}/role")
async def change_role(user_id: str, payload: dict, request: Request,
                     admin: dict = Depends(require_admin)):
    """Change a user's account_level. Audit-logged."""
    new_role = payload.get("account_level", "")
    if new_role not in ("student", "teacher", "admin"):
        raise HTTPException(400, "Invalid role")

    async with AsyncSessionLocal() as db:
        student = (await db.execute(select(Student).where(Student.id == uuid.UUID(user_id)))).scalar_one_or_none()
        if not student:
            raise HTTPException(404, "User not found")
        old_role = student.account_level
        student.account_level = new_role
        await db.commit()

    audit_log("role_change", actor=admin["sub"], target=user_id,
             request=request, outcome="success",
             details=f"{old_role} → {new_role}")
    return {"status": "ok", "user_id": user_id, "account_level": new_role}

@router.post("/users/{user_id}/disable")
async def disable_user(user_id: str, request: Request, admin: dict = Depends(require_admin)):
    """Mark a user inactive."""
    async with AsyncSessionLocal() as db:
        student = (await db.execute(select(Student).where(Student.id == uuid.UUID(user_id)))).scalar_one_or_none()
        if not student:
            raise HTTPException(404, "User not found")
        student.is_active = 0
        await db.commit()

    audit_log("user_disabled", actor=admin["sub"], target=user_id, request=request)
    return {"status": "ok"}

@router.delete("/courses/{course_id}")
async def delete_course(course_id: str, admin: dict = Depends(require_admin)):
    """Delete a course (cascade)."""
    async with AsyncSessionLocal() as db:
        # Cascade delete via FK
        await db.execute(text(f'DELETE FROM courses WHERE id = :cid'), {"cid": course_id})
        await db.commit()
    # Also wipe Qdrant chunks for this course
    rag = deps.get_rag()
    rag.delete_course(course_id)
    return {"status": "ok"}

```

9. routes/analytics.py

Rôle : endpoints analytics (KPIs, dashboards).

```
router = APIRouter(prefix="/analytics")

@router.get("/kpi")
async def get_kpi(hours: int = 24):
    """Aggregate KPIs over the last N hours."""
    from observability.analytics import get_analytics
    a = get_analytics()
    return a.kpi_summary(hours=hours)

@router.get("/latency")
async def get_latency(hours: int = 24):
    """Latency distribution (p50, p95, p99)."""
    from observability.analytics import get_analytics
    a = get_analytics()
    return a.latency_distribution(hours=hours)

@router.get("/by-language")
async def by_language():
    from observability.analytics import get_analytics
    return get_analytics().by_language()

@router.get("/by-subject")
async def by_subject():
    from observability.analytics import get_analytics
    return get_analytics().by_subject()

@router.get("/full-report")
async def full_report():
    """Complete analytics for the dashboard."""
    from observability.analytics import get_analytics
    return get_analytics().full_report()

@router.get("/clickhouse/query")
async def clickhouse_query(sql: str, admin: dict = Depends(require_admin)):
    """Raw ClickHouse query (admin only). Read-only sanity check enforced."""
    if not sql.strip().upper().startswith("SELECT"):
        raise HTTPException(400, "Only SELECT queries allowed")
    from observability.analytics import get_analytics
    a = get_analytics()
    if not a._use_ch:
        raise HTTPException(503, "ClickHouse not configured")
    rows = a._ch.query(sql).result_rows
    return {"rows": rows[:1000]} # cap output
```

10. routes/dashboard_services.py

Rôle : endpoints qui agrègent l'état des services pour le dashboard frontend.

```
router = APIRouter(prefix="/dashboard")

@router.get("/services")
async def get_services_status():
    """Returns health status of all backend services."""
    services = {}

    # Postgres
```



```

try:
    from database.init_db import check_db_connection
    services["postgres"] = "up" if await check_db_connection() else "down"
except Exception:
    services["postgres"] = "down"

# Redis
try:
    from pedagogy.dialogue import get_redis
    r = await get_redis()
    await r.ping()
    services["redis"] = "up"
except Exception:
    services["redis"] = "down"

# Qdrant
try:
    from qdrant_client import QdrantClient
    c = QdrantClient(host=Config.QDRANT_HOST, port=Config.QDRANT_PORT, timeout=2)
    c.get_collections()
    services["qdrant"] = "up"
except Exception:
    services["qdrant"] = "down"

# Ollama
try:
    import requests
    r = requests.get(f"{Config.OLLAMA_URL}/api/tags", timeout=2)
    services["ollama"] = "up" if r.status_code == 200 else "degraded"
except Exception:
    services["ollama"] = "down"

# Elasticsearch
try:
    from storage.transcript_search import get_search_index
    si = get_search_index()
    services["elasticsearch"] = "up" if si._use_es else "fallback_memory"
except Exception:
    services["elasticsearch"] = "down"

# MinIO
try:
    from storage.media_storage import get_storage
    st = get_storage()
    services["minio"] = "up" if st._use_minio else "fallback_local"
except Exception:
    services["minio"] = "down"

# ClickHouse
try:
    from observability.analytics import get_analytics
    a = get_analytics()
    services["clickhouse"] = "up" if a._use_ch else "fallback_csv"
except Exception:
    services["clickhouse"] = "down"

return {"services": services, "timestamp": time.time()}

@router.get("/rag/stats")
async def get_rag_stats():
    rag = deps.get_rag()
    return rag.get_stats()

@router.get("/llm/stats")
async def get_llm_stats():
    from ai.llm_router import get_default_router
    return get_default_router().stats.snapshot()

```

11. routes/health.py

```
router = APIRouter()

@router.get("/health")
async def health_check():
    """Lightweight health check (used by Docker, load balancers)."""
    return {"status": "ok", "service": "smart_teacher", "timestamp": time.time()}

@router.get("/ready")
async def readiness():
    """Readiness probe : are all critical services up?"""
    try:
        from database.init_db import check_db_connection
        from pedagogy.dialogue import get_redis
        if not await check_db_connection():
            raise HTTPException(503, "Postgres down")
        r = await get_redis()
        await r.ping()
        return {"status": "ready"}
    except Exception as exc:
        raise HTTPException(503, f"Not ready: {exc}")

@router.get("/diagnostics")
async def diagnostics_endpoint():
    """Full diagnostics report (admin-style, less restrictive)."""
    from core.diagnostics import full_diagnostics
    return await full_diagnostics()
```

12. routes/learning_gain.py

Rôle : endpoints pour pre-test / post-test (mesure du learning gain).

```
router = APIRouter(prefix="/learning-gain")

class LearningGainCreate(BaseModel):
    course_id: str
    test_type: str # "pretest" | "posttest"
    questions: list[dict]
    student_answers: list[dict]

@router.post("/test")
async def submit_test(payload: LearningGainCreate, user: dict = Depends(get_current_user)):
    """Submit a pre-test or post-test for grading."""
    student_id = uuid.UUID(user["sub"])

    # 1. Grade each answer (LLM-graded)
    from pedagogy.practice_engine import PracticeEngine
    pe = PracticeEngine(rag=deps.get_rag(), brain=deps.get_brain())
    graded = []
    correct_count = 0
    for q, a in zip(payload.questions, payload.student_answers):
        result = await pe.grade_attempt(q, a.get("text", ""))
        graded.append({**q, "student_answer": a, "grading": result})
        if result.get("correct"):
            correct_count += 1
```

```

score = correct_count / max(1, len(payload.questions))

# 2. Persist
async with AsyncSessionLocal() as db:
    test = LearningGainTest(
        student_id=student_id,
        course_id=uuid.UUID(payload.course_id),
        test_type=payload.test_type,
        score=score,
        questions=graded,
        taken_at=datetime.utcnow(),
    )
    db.add(test)
    await db.commit()

return {"score": score, "correct": correct_count, "total": len(payload.questions),
        "graded": graded}

@router.get("/student/{student_id}/gain")
async def compute_gain(student_id: str, course_id: str):
    """Compute learning gain = (post - pre) / (max - pre).

    Hake's normalized gain (Hake 1998) :
    g = (post% - pre%) / (100% - pre%)
    Returns NaN if pre% = 100%.
    """
    async with AsyncSessionLocal() as db:
        stmt = select(LearningGainTest).where(
            LearningGainTest.student_id == uuid.UUID(student_id),
            LearningGainTest.course_id == uuid.UUID(course_id),
        ).order_by(LearningGainTest.taken_at.asc())
        tests = (await db.execute(stmt)).scalars().all()

    pre = next((t for t in tests if t.test_type == "pretest"), None)
    post = next((t for t in tests if t.test_type == "posttest"), None)
    if not pre or not post:
        return {"gain": None, "reason": "missing_pre_or_post"}

    pre_pct = pre.score
    post_pct = post.score
    if pre_pct >= 1.0:
        return {"gain": None, "pre": pre_pct, "post": post_pct, "reason": "ceiling"}

    gain = (post_pct - pre_pct) / (1.0 - pre_pct)
    interpretation = "low" if gain < 0.3 else "medium" if gain < 0.7 else "high"
    return {
        "gain": round(gain, 3),
        "pre": round(pre_pct, 3),
        "post": round(post_pct, 3),
        "interpretation": interpretation,
    }

```

13. routes/practice.py

Rôle : endpoints pour génération + soumission de questions de pratique.

```
router = APIRouter(prefix="/practice")
```

```

@router.get("/question")
async def get_practice_question(course_id: str, concept_name: str = None,
                                difficulty: str = "balanced",
                                user: dict = Depends(get_current_user)):
    """Generate a practice question.
    If concept_name not given → pick the most due concept (FSRS) for this student.

```

```

"""
student_id = uuid.UUID(user["sub"])

if not concept_name:
    # Pick most due concept
    from pedagogy.review_scheduler import ReviewScheduler
    due = await ReviewScheduler.list_due(student_id, course_id, limit=1)
    if not due:
        raise HTTPException(404, "No due concepts")
    concept_name = due[0]["concept_name"]

from pedagogy.practice_engine import PracticeEngine
pe = PracticeEngine(rag=deps.get_rag(), brain=deps.get_brain())
question = await pe.generate_question(
    concept_name=concept_name,
    course_id=course_id,
    difficulty=difficulty,
    question_type="open",
)
if not question:
    raise HTTPException(500, "Could not generate question")
return question

class PracticeSubmit(BaseModel):
    course_id: str
    concept_name: str
    question: dict
    answer: str

@router.post("/submit")
async def submit_practice(payload: PracticeSubmit, user: dict = Depends(get_current_user)):
    """Grade attempt + update FSRS + update mastery."""
    student_id = uuid.UUID(user["sub"])

    # 1. Grade
    from pedagogy.practice_engine import PracticeEngine
    pe = PracticeEngine(rag=deps.get_rag(), brain=deps.get_brain())
    result = await pe.grade_attempt(payload.question, payload.answer)

    # 2. Map grading → FSRS rating
    if not result.get("correct"):
        rating = "again"
    elif result.get("score", 0) < 0.6:
        rating = "hard"
    elif result.get("score", 0) < 0.9:
        rating = "good"
    else:
        rating = "easy"

    # 3. Update FSRS
    from pedagogy.review_scheduler import ReviewScheduler
    next_due = await ReviewScheduler.update_after_practice(student_id, payload.concept_name, rating)

    # 4. Update mastery on the underlying ideas
    from pedagogy.knowledge_graph import get_or_build
    kg = get_or_build(deps.get_rag())
    concept = kg.get_concept(payload.concept_name)
    if concept:
        from pedagogy.mastery_repo import MasteryRepo
        for idea_id in list(concept.idea_ids)[:3]: # cap to 3
            await MasteryRepo.update(student_id, payload.course_id, idea_id,
                                    is_confusion=not result.get("correct"))

    # 5. Persist attempt
    async with AsyncSessionLocal() as db:
        attempt = PracticeAttempt(
            student_id=student_id,
            course_id=uuid.UUID(payload.course_id),
            concept_name=payload.concept_name,

```

```

        question_text=payload.question.get("text", ""),
        student_answer=payload.answer,
        correct=result.get("correct"),
        score=result.get("score", 0.0),
        feedback=result.get("feedback", ""),
        rating=rating,
        attempted_at=datetime.utcnow(),
    )
    db.add(attempt)
    await db.commit()

return {
    "correct":    result.get("correct"),
    "score":      result.get("score"),
    "feedback":   result.get("feedback"),
    "rating":     rating,
    "next_due":   next_due.isoformat() if next_due else None,
}

```

14. routes/rest.py

Rôle : endpoint REST `/ask` one-shot (pas WebSocket).

```

router = APIRouter()

class AskRequest(BaseModel):
    question: str
    course_id: Optional[str] = None
    session_id: Optional[str] = None
    language: str = "fr"

@router.post("/ask")
async def ask_question(payload: AskRequest, user: dict = Depends(get_current_user)):
    """One-shot Q&A REST. No streaming, no audio."""
    from handlers.rest_routes import handle_ask
    return await handle_ask(
        question=payload.question,
        course_id=payload.course_id,
        session_id=payload.session_id or str(uuid.uuid4()),
        language=payload.language,
        student_id=user["sub"],
    )

```

15. routes/search.py

Rôle : recherche full-text dans les transcripts.

```

router = APIRouter()

@router.get("/search")
async def search_transcripts(query: str, language: str = "", course_id: str = "",
                             role: str = "", limit: int = 20):
    """Search Q&A transcripts via Elasticsearch (or memory fallback)."""
    from storage.transcript_search import get_search_index
    si = get_search_index()
    results = si.search(query, language=language, course_id=course_id,
                        role=role, limit=limit)

```

```

return {
    "query": query,
    "total": len(results),
    "results": results,
}

```

16. routes/voice.py

Rôle : endpoints pour upload de fichier audio (alternative au WS streaming).

```

router = APIRouter(prefix="/voice")

@router.post("/transcribe")
async def transcribe_audio(file: UploadFile = File(...), language: str = Form("fr")):
    """Upload audio file → transcribe + return text."""
    audio_bytes = await file.read()
    # Decode to numpy float32
    audio = decode_audio_bytes(audio_bytes, sr=Config.SAMPLE_RATE)

    transcriber = deps.get_transcriber()
    text, stt_time, lang, lang_prob, audio_duration = transcriber.transcribe(
        audio, force_language=language)

    return {
        "text": text,
        "language": lang,
        "language_prob": round(lang_prob, 2),
        "audio_duration": round(audio_duration, 2),
        "stt_time": round(stt_time, 2),
        "rtf": round(stt_time / audio_duration, 2) if audio_duration > 0 else 0,
    }

@router.post("/synthesize")
async def synthesize_text(payload: dict, user: dict = Depends(get_current_user)):
    """Generate TTS audio from text. Returns MP3 bytes."""
    text = payload.get("text", "")
    language = payload.get("language", "fr")
    rate = payload.get("rate", "+0%")

    voice = deps.get_voice()
    audio_bytes, duration, engine, voice_name, mime = await voice.generate_audio_async(
        text, language_code=language, rate=rate)

    if not audio_bytes:
        raise HTTPException(500, "TTS failed")

    return Response(content=audio_bytes, media_type=mime or "audio/mpeg",
        headers={"X-Audio-Duration": str(duration)})

```

17. routes/student_state.py

Rôle : endpoint riche pour le state observability d'un étudiant.

```

@router.get("/student/me/state")
async def get_state(user: dict = Depends(get_current_user)):
    """Aggregated student state for the UI."""
    student_id = uuid.UUID(user["sub"])
    # Voir Part 2 §49 pour le détail. Couvre :

```

```
# - mastery stats
# - confusion count
# - FSRS due reviews
# - knowledge snapshot (last course)
# - engagement
...
```

18. handlers/rest_routes.py

Rôle : business logic derrière les endpoints REST (séparée des routes pour testabilité).

```
async def handle_ask(question: str, course_id: str, session_id: str,
                    language: str, student_id: str) -> dict:
    """One-shot Q&A : same pipeline as WS but synchronous, no streaming."""
    from agentic.qa.graph import build_qa_graph
    rag = deps.get_rag()
    brain = deps.get_brain()
    graph = build_qa_graph(brain, rag)

    state = TutorState(
        question=question, raw_question=question,
        language=language, course_id=course_id,
        session_id=session_id, student_id=student_id,
    )
    final_state = await graph.ainvoke(state)

    return {
        "answer":          final_state.answer,
        "confidence":      final_state.confidence,
        "supporting_chunks": [{ "id": c.get("id"), "text": c.get("content", "")[:300]}
                               for c in final_state.citations],
        "intent":          final_state.intent.type if final_state.intent else "unknown",
        "performance":     { "total_time": final_state.timings.get("total", 0)},
    }

async def handle_session_profile(session_id: str):
    from pedagogy.personalization.profile import get_or_create_profile
    profile = await get_or_create_profile(session_id)
    if not profile:
        raise HTTPException(404, "Profile not found")
    return {"status": "ok", "profile": profile}

async def handle_session_profile_update(session_id: str, payload: dict):
    from pedagogy.personalization.profile import update_profile, get_or_create_profile
    await get_or_create_profile(session_id)
    updated = await update_profile(session_id, payload)
    return {"status": "ok", "profile": updated}

async def handle_session_tts_params(session_id: str, confusion_score: float = 0.0):
    from pedagogy.personalization.tts_adapter import compute_tts_params
    params = await compute_tts_params(session_id, confusion_score=confusion_score)
    return {"status": "ok", "tts_params": params}
```

19. pedagogy/concept_types.py

Rôle : dataclasses pour les concepts (utilisé par concept_from_titles + extractors legacy).

```

@dataclass
class Concept:
    """Concept extracted from course content. Used during ingestion before
    being converted to ConceptInfo for the KG."""
    label: str # snake_case unique id
    name: str # human-readable
    keywords: list[str] = field(default_factory=list)
    chunk_ids: set[str] = field(default_factory=set)
    chapter_idx: set[int] = field(default_factory=set)
    score: float = 0.0 # importance
    description: str = ""
    bloom_level: str = ""
    canonical_name: str = ""

    def to_dict(self) -> dict:
        return {**asdict(self),
                "chunk_ids": list(self.chunk_ids),
                "chapter_idx": sorted(self.chapter_idx)}

    def to_concept_info(self, course_id: str) -> "ConceptInfo":
        """Bridge : Concept (extractor staging) → ConceptInfo (KG)."""
        from pedagogy.knowledge_graph.graph import ConceptInfo
        return ConceptInfo(
            name=self.label,
            display_name=self.name,
            canonical_name=self.canonical_name,
            description=self.description,
            bloom_level=self.bloom_level,
            course_id=course_id,
            chapter_idx=set(self.chapter_idx),
            score=self.score,
            idea_ids=set(self.chunk_ids),
        )

def _slugify(name: str) -> str:
    """Slugify for use as concept label (snake_case ASCII)."""
    s = name.lower()
    s = unicodedata.normalize("NFKD", s).encode("ascii", "ignore").decode("ascii")
    s = re.sub(r"[^a-z0-9]+", "_", s).strip("_")
    return s[:64] or "concept"

```

20. pedagogy/course_analyzer.py

Rôle : analyse high-level d'un cours (langue, niveau, topics dominants).

```

async def analyze_course(course_id: str, rag) -> dict:
    """Returns {language, level, topics, n_chapters, ...}."""
    async with AsyncSessionLocal() as db:
        course = (await db.execute(
            select(Course).where(Course.id == _coerce_uuid(course_id))
        )).scalar_one_or_none()
        if not course:
            return {"language": "fr", "level": "lycée", "topics": [], "n_chapters": 0}

        chapters = (await db.execute(
            select(Chapter).where(Chapter.course_id == course.id).order_by(Chapter.order)
        )).scalars().all()

        # Sample content from first 3 chapters
        sample_text = ""
        for ch in chapters[:3]:
            sections = (await db.execute(
                select(Section).where(Section.chapter_id == ch.id).limit(5)
            )).scalars().all()
            for s in sections:

```



```

        sample_text += " " + (s.content or "")

# Detect language
language = course.language or detect_lang_text(sample_text[:2000])

# Estimate level (could be LLM-based or keyword heuristic)
level = course.level
if not level:
    level = _heuristic_level(sample_text)

# Top topics via KeyBERT or KG concepts
try:
    from pedagogy.knowledge_graph import get_or_build, ensure_concepts_loaded
    kg = get_or_build(rag)
    concepts = await ensure_concepts_loaded(rag, course_id, enrich=False)
    topics = [c.name for c in concepts[:10]]
except Exception:
    topics = []

log.info("■ Course analyzed: lang=%s level=%s topics=%d chapters=%d",
        language, level, len(topics), len(chapters))
return {
    "language": language,
    "level": level,
    "topics": topics,
    "n_chapters": len(chapters),
    "course_domain": course.domain,
}

def _heuristic_level(text: str) -> str:
    """Quick heuristic : avg sentence length + complex word ratio."""
    sentences = re.split(r"[.!?]+", text[:5000])
    avg_words = sum(len(s.split()) for s in sentences) / max(1, len(sentences))
    if avg_words < 10:
        return "collège"
    if avg_words < 20:
        return "lycée"
    return "université"

```

21. pedagogy/slide_sync.py

Rôle : synchronise le curseur narration avec l'index de slide affiché.

```

def cursor_to_slide_index(narration: str, cursor: int, n_slides: int) -> int:
    """Convert char cursor → slide index (linear approximation)."""
    if not narration or n_slides <= 0:
        return 0
    progress = cursor / max(1, len(narration))
    return min(n_slides - 1, int(progress * n_slides))

def slide_index_to_cursor(narration: str, slide_idx: int, n_slides: int) -> int:
    """Inverse : slide index → char cursor."""
    if n_slides <= 0:
        return 0
    progress = slide_idx / n_slides
    return int(progress * len(narration))

async def emit_slide_sync(send, narration_text: str, cursor: int,
                        slide_index: int, slide_path: str, slide_title: str):
    """Send slide_sync WS event so UI updates highlighted slide."""
    log.info("■ slide sync | type=image ch=%d sec=%d title=%r progress=%d%%",
            chapter_idx, slide_index, slide_title,
            int(cursor / max(1, len(narration_text)) * 100))

```

```

await send({
    "type": "slide_sync",
    "slide_index": slide_index,
    "slide_path": slide_path,
    "slide_title": slide_title,
    "cursor": cursor,
    "total_chars": len(narration_text),
})

```

22. pedagogy/student_model.py

Rôle : modèle étudiant agrégé (pour la dashboard teacher).

```

@dataclass
class StudentModel:
    student_id: str
    courses: list[dict]
    total_attempts: int
    avg_mastery: float
    weak_concepts: list[str]
    due_reviews: int
    learning_style: str
    last_active: datetime

    @classmethod
    async def build(cls, student_id: str) -> "StudentModel":
        """Build from Postgres + Redis."""
        async with AsyncSessionLocal() as db:
            # Courses seen
            courses_stmt = select(LearningSession.course_id, func.count().label("n_sessions")).where(
                LearningSession.student_id == _coerce_uuid(student_id)
            ).group_by(LearningSession.course_id)
            courses_rows = (await db.execute(courses_stmt)).all()
            courses = [{"course_id": str(r.course_id), "n_sessions": r.n_sessions}
                        for r in courses_rows]

            # Mastery stats
            mastery_stmt = select(
                func.count(StudentMastery.id),
                func.avg(StudentMastery.score),
            ).where(StudentMastery.student_id == _coerce_uuid(student_id))
            n_attempts, avg_mastery = (await db.execute(mastery_stmt)).first()

            # Weak concepts (mastery < 0.4 + recently attempted)
            weak_stmt = select(StudentMastery.idea_id).where(
                StudentMastery.student_id == _coerce_uuid(student_id),
                StudentMastery.score < 0.4,
            ).order_by(StudentMastery.last_seen_at.desc()).limit(10)
            weak_idea_ids = [r[0] for r in (await db.execute(weak_stmt)).all()]

            # Due reviews
            from pedagogy.review_scheduler import ReviewScheduler
            due = await ReviewScheduler.list_due(_coerce_uuid(student_id), limit=100)

            # Learning style from Redis profile
            from pedagogy.personalization.profile import get_or_create_profile
            profile = await get_or_create_profile(student_id)
            learning_style = (profile.get("learning_style_posterior") or {}).get("dominant", "verbal")

        return cls(
            student_id=str(student_id),
            courses=courses,
            total_attempts=n_attempts or 0,
            avg_mastery=float(avg_mastery or 0.5),
            weak_concepts=weak_idea_ids,
            due_reviews=len(due),

```

```

        learning_style=learning_style,
        last_active=datetime.utcnow(),
    )

```

23. pedagogy/personalization/learning_style/heuristic.py

Rôle : fallback heuristique pour détecter le style si Bayes pas suffisant de data.

```

HEURISTIC_KEYWORDS = {
    "visual":      {"voir", "image", "schéma", "graphe", "diagramme", "imaginer"},
    "verbal":      {"définir", "expliquer", "décrire", "écouter"},
    "kinesthetic": {"faire", "manipuler", "essayer", "construire"},
    "read_write": {"lire", "écrire", "noter", "résumer"},
}

def detect_style_heuristic(history: list[dict], lang: str = "fr") -> str:
    """Count keyword occurrences across student questions, return dominant style."""
    counts = {k: 0 for k in HEURISTIC_KEYWORDS}
    for turn in history:
        if turn.get("role") != "user":
            continue
        text = turn.get("text", "").lower()
        for style, keywords in HEURISTIC_KEYWORDS.items():
            for kw in keywords:
                if kw in text:
                    counts[style] += 1
    return max(counts, key=counts.get)

```

24. pedagogy/personalization/bandit/log_extractor.py

Rôle : extrait les logs bandit pour offline learning.

```

def extract_bandit_logs(start_ts: float, end_ts: float, output_path: str = None) -> list[dict]:
    """Parse server logs → list of {session, arm, context, reward, ts}.

    Used by `scripts/extract_bandit_logs.py` to build datasets for
    Phase 3 offline RL.
    """
    log_files = list(Path(Config.LOGS_DIR).glob("*.log"))
    samples = []

    BANDIT_RE = re.compile(
        r"bandit\.end_turn session=(?P<sid>\S+) arm=(?P<arm>\S+) "
        r"reward=(?P<reward>[\d.]+) "
    )
    CONTEXT_RE = re.compile(
        r"bandit START \| session=(?P<sid>\S+) ctx_bucket=(?P<bucket>\S+) "
        r"\| strategy=\S+ speech_rate=\S+"
    )

    for log_file in log_files:
        with open(log_file, encoding="utf-8") as f:
            for line in f:
                ts = parse_log_timestamp(line)
                if ts is None or not (start_ts <= ts <= end_ts):

```

```

        continue
    m_ctx = CONTEXT_RE.search(line)
    if m_ctx:
        # Save context, will pair with reward later
        ...
    m_reward = BANDIT_RE.search(line)
    if m_reward:
        samples.append({
            "session_id": m_reward.group("sid"),
            "arm": m_reward.group("arm"),
            "reward": float(m_reward.group("reward")),
            "ts": ts,
            # context filled by pairing logic
        })

if output_path:
    with open(output_path, "w") as f:
        json.dump(samples, f, indent=2)
return samples

```

25. pedagogy/personalization/bandit/offline.py

Rôle : offline learning des arms (rebuild posteriors à partir des logs historiques).

```

async def rebuild_posteriors_from_logs(start_ts: float, end_ts: float,
                                       course_id: str = None) -> dict:
    """Rebuild Beta( $\alpha$ ,  $\beta$ ) posteriors from historical interactions.

    Useful pour :
    - Reset posteriors corrompus
    - Bootstrap nouvelles arms
    - A/B test sur paramètres reward weights
    """
    samples = extract_bandit_logs(start_ts, end_ts)
    if course_id:
        samples = [s for s in samples if s.get("course_id") == course_id]

    # Group by (course, bucket, arm)
    grouped = defaultdict(lambda: {"alpha": 1.0, "beta": 1.0, "n": 0})
    for s in samples:
        key = (s["course_id"], s.get("bucket", "default"), s["arm"])
        grouped[key]["alpha"] += s["reward"]
        grouped[key]["beta"] += (1 - s["reward"])
        grouped[key]["n"] += 1

    # Persist new posteriors
    from pedagogy.personalization.bandit.repo import BanditRepo
    repo = BanditRepo()
    for (cid, bucket, arm_name), state in grouped.items():
        await repo.set_posterior(cid, bucket, arm_name, state["alpha"], state["beta"], state["n"])

    log.info("Rebuilt %d (course, bucket, arm) posteriors from %d samples",
            len(grouped), len(samples))
    return {"n_samples": len(samples), "n_posteriors": len(grouped)}

```

26. pedagogy/personalization/bandit/simulat or.py

Rôle : simulator pour A/B test offline des stratégies.

```
class StudentSimulator:
    """Simulates a student response to a teaching strategy.

    Used to evaluate bandit performance before deploying to real users.
    Phase 2 in the personalization roadmap.
    """
    def __init__(self, true_preferences: dict, noise: float = 0.1):
        self.true_preferences = true_preferences # {"socratic:slow": 0.8, ...}
        self.noise = noise
        self.rng = random.Random(42)

    def respond(self, arm_name: str) -> float:
        """Returns reward  $\in [0, 1]$  based on simulated preference + noise."""
        true_reward = self.true_preferences.get(arm_name, 0.5)
        noisy = true_reward + self.rng.gauss(0, self.noise)
        return max(0.0, min(1.0, noisy))

async def simulate_bandit(n_turns: int = 1000, n_arms: int = 18,
                          true_best_arm: str = "socratic:slow") -> dict:
    """Run bandit on simulated student. Returns regret + best arm convergence."""
    arms_state = {f"arm_{i}": ArmState() for i in range(n_arms)}
    arms_state[true_best_arm] = ArmState()

    sim = StudentSimulator(true_preferences={true_best_arm: 0.85, "default": 0.4})
    cumulative_regret = 0
    pulls = defaultdict(int)
    rewards = defaultdict(list)

    for t in range(n_turns):
        chosen = select_arm(arms_state, sim.rng)
        reward = sim.respond(chosen)
        update_arm(arms_state, chosen, reward)
        pulls[chosen] += 1
        rewards[chosen].append(reward)
        cumulative_regret += sim.true_preferences.get(true_best_arm, 0.85) - reward

    return {
        "cumulative_regret": cumulative_regret,
        "best_arm_pulls": pulls.get(true_best_arm, 0),
        "best_arm_pct": pulls.get(true_best_arm, 0) / n_turns,
        "arm_distribution": dict(pulls),
    }
```

27. pedagogy/knowledge_graph/persistence.py

Rôle : sauvegarde / chargement disque des concepts pour éviter de re-extraire à chaque restart.

```
_CACHE_PATH = Path("data") / "kg_concepts_cache.json"

def save_concepts(course_id: str, concepts: list[ConceptInfo], n_docs: int) -> None:
    """Persist concepts on disk for fast cross-restart load.

    Cache invalidated when n_docs (RAG corpus size) changes – that's the
    cheap signal for "ingestion happened, concepts may need rebuilding".
    """
    _CACHE_PATH.parent.mkdir(parents=True, exist_ok=True)

    # Load existing cache
    cache = {}
```

```

if _CACHE_PATH.exists():
    try:
        cache = json.loads(_CACHE_PATH.read_text(encoding="utf-8"))
    except Exception:
        cache = {}

# Update entry for this course
cache[course_id] = {
    "n_docs": n_docs,
    "concepts": [c.to_dict() for c in concepts],
    "saved_at": time.time(),
}

_CACHE_PATH.write_text(json.dumps(cache, ensure_ascii=False), encoding="utf-8")
log.info("KG concepts saved : course=%s n=%d (cache=%s)",
        course_id[:8], len(concepts), _CACHE_PATH.name)

def load_concepts(course_id: str, n_docs: int) -> Optional[list[ConceptInfo]]:
    """Load cached concepts if n_docs matches (= no re-ingestion since save)."""
    if not _CACHE_PATH.exists():
        return None
    try:
        cache = json.loads(_CACHE_PATH.read_text(encoding="utf-8"))
    except Exception:
        return None

    entry = cache.get(course_id)
    if not entry:
        return None
    if entry.get("n_docs") != n_docs:
        log.debug("KG concepts cache miss : n_docs changed (%d → %d)",
                entry.get("n_docs"), n_docs)
        return None

    log.info("KG concepts loaded from cache : course=%s n=%d (n_docs=%d match)",
            course_id[:8], len(entry["concepts"]), n_docs)
    return [
        ConceptInfo(
            name=c["name"], display_name=c.get("display_name", ""),
            canonical_name=c.get("canonical_name", ""),
            description=c.get("description", ""),
            bloom_level=c.get("bloom_level", ""),
            course_id=c.get("course_id", ""),
            chapter_idxs=set(c.get("chapter_idxs", [])),
            score=c.get("score", 0.0),
            idea_ids=set(c.get("idea_ids", [])),
        )
        for c in entry["concepts"]
    ]

```

28. core/diagnostics.py

Rôle : health checks détaillés au boot + endpoint diagnostics.

```

def print_service_health():
    """Boot-time service health summary."""
    log.info("■ Diagnostic démarrage des services de données :")

    # Elasticsearch
    try:
        from storage.transcript_search import get_search_index
        si = get_search_index()
        if si._use_es:
            log.info("    • Elasticsearch : ■ connecté")
        else:

```

```

        log.info("    • Elasticsearch : ■■ recherche en mémoire (fallback actif)")
except Exception:
    log.info("    • Elasticsearch : ■ erreur")

# MinIO
try:
    from storage.media_storage import get_storage
    st = get_storage()
    if st._use_minio:
        log.info("    • MinIO : ■ %s/%s", Config.MINIO_ENDPOINT, Config.MINIO_BUCKET)
    else:
        log.info("    • MinIO : ■■ stockage local (%s)", Config.LOCAL_MEDIA_DIR)
except Exception:
    log.info("    • MinIO : ■ erreur")

# ClickHouse
try:
    from observability.analytics import get_analytics
    a = get_analytics()
    if a._use_ch:
        log.info("    • ClickHouse : ■ %s.%s", Config.CLICKHOUSE_DB, "learning_events")
    else:
        log.info("    • ClickHouse : ■■ analytics CSV + mémoire")
except Exception:
    log.info("    • ClickHouse : ■ erreur")

# Redis (toujours requis)
try:
    import redis as redis_sync
    r = redis_sync.Redis(host=Config.REDIS_HOST, port=Config.REDIS_PORT, socket_connect_timeout=3)
    r.ping()
    log.info("    • Redis : ■ connecté (%s:%d)", Config.REDIS_HOST, Config.REDIS_PORT)
except Exception as exc:
    log.error("    • Redis : ■ INDISPONIBLE (%s)", exc)

async def full_diagnostics() -> dict:
    """Endpoint /diagnostics : full status report."""
    return {
        "postgres":    await _ping_pg(),
        "redis":        await _ping_redis(),
        "qdrant":       _ping_qdrant(),
        "ollama":       _ping_ollama(),
        "elasticsearch": _es_status(),
        "minio":        _minio_status(),
        "clickhouse":   _clickhouse_status(),
        "rag_stats":    deps.get_rag().get_stats(),
        "llm_stats":    get_default_router().stats.snapshot(),
        "config_snapshot": {
            "DISABLE_OPENAI": Config.DISABLE_OPENAI,
            "GROQ_ENABLED":    bool(Config.GROQ_API_KEY),
            "RAG_USE_IDEA_CHUNKING": Config.RAG_USE_IDEA_CHUNKING,
            "KG_DISABLE_LLM_ENRICH": Config.KG_DISABLE_LLM_ENRICH,
            "TTS_PROVIDER":    Config.TTS_PROVIDER,
            "WHISPER_MODEL":    Config.WHISPER_MODEL_SIZE,
        },
    },
}

```

29. core/domains_config.py

Rôle : config des domaines pédagogiques (informatique, math, etc.) + auto-detect.

```

DOMAINS = {
    "informatique": {
        "name": "Informatique",
        "description": "computer science topics including algorithms, programming, AI, ...",
    },

```

```

        "keywords":    ["algorithm", "code", "python", "data", "machine learning", "ri", ...],
    },
    "mathematics": {
        "name":        "Mathématiques",
        "description":  "mathematical topics including calculus, algebra, statistics, ...",
        "keywords":    ["équation", "intégrale", "matrice", "vecteur", "fonction", ...],
    },
    "general": {
        "name":        "Général",
        "description":  "specialty subjects",
        "keywords":    [],
    },
}

```

```

def auto_detect_course(file_path: str) -> tuple[str, str]:
    """Detect (domain, course_slug) from filename + folder path.

```

Examples:

```

'courses/informatique/recherche_information/Chapitre 1.pdf'
→ ('informatique', 'recherche_information')
'random.pdf' → ('general', 'generic')
"""

```

```

p = Path(file_path)
parts = list(p.parts)
if "courses" in parts:
    idx = parts.index("courses")
    if len(parts) > idx + 2:
        return parts[idx + 1], parts[idx + 2]
return ("general", "generic")

```

```

def classify_course_via_llm(file_path: str, max_pages: int = 2) -> Optional[dict]:
    """Last-resort LLM classification of an unknown course."""
    # Extract first 2 pages of text
    sample_text = _extract_first_pages(file_path, max_pages)

    # Build LLM prompt
    domains_list = "\n".join([f"- {k}: {v['description']}" for k, v in DOMAINS.items()])
    prompt = (
        f"Classify this course content into a domain + a course slug.\n\n"
        f"Content (first 2 pages):\n{sample_text[:3000]}\n\n"
        f"Domains : \n{domains_list}\n\n"
        f"JSON : {{\n  \"domain\": \"...\", \n  \"course\": \"snake_case_short\", \n  \"title\": \"...\" }}"
    )

    # Use Groq when OpenAI off
    if Config.DISABLE_OPENAI and Config.GROQ_API_KEY:
        log.info("■ LLM classify : Groq activ   (model=%s)", Config.GROQ_MODEL)
        try:
            from langchain_openai import ChatOpenAI
            from langchain_core.messages import HumanMessage
            llm = ChatOpenAI(
                model=Config.GROQ_MODEL,
                api_key=Config.GROQ_API_KEY,
                base_url=Config.GROQ_BASE_URL,
                temperature=0.0, max_tokens=300, max_retries=0,
            )
            response = llm.invoke([HumanMessage(content=prompt)])
            return _parse_classification(response.content)
        except Exception as exc:
            log.warning(f"■ Groq classify failed: {exc} → fallback Ollama")

    # ... (Ollama fallback same as before)

```

30. ai/prompt_rules.py

Rôle : règles partagées entre tous les prompts (cross-language, math notation).

```
CROSSLANG_RULE_FR = (
    "Si le matériel du cours est dans une langue différente de ta réponse, "
    "comprends la source naturellement et réponds dans la langue demandée ; "
    "à la première mention d'un terme technique clé, conserve l'écriture "
    "originale de la slide entre parenthèses."
)

CROSSLANG_RULE_EN = (
    "If the course material is in a different language than your answer, "
    "parse the source naturally and answer in the requested language; "
    "on first mention of a key technical term keep the slide's original "
    "wording in parentheses."
)
```

Single source of truth → utilisée par `ai/llm.get_system_prompt`, `ai/llm.get_presentation_prompt`, `agentic/qa/responder._build_qa_prompt`, `agentic/qa/rewriter._build_rewrite_prompt`.

31. deps.py

Rôle : dependency injection — singletons partagés (rag, voice, brain, dialogue, transcriber).

```
_rag = None
_voice = None
_brain = None
_dialogue = None
_transcriber = None
_csv_logger = None
_stt_logger = None

active_vfms: dict[str, VoiceStateMachine] = {}      # debug : registry des FSM actives

def init_singletons():
    """Called once at boot from main.py lifespan."""
    global _rag, _voice, _brain, _dialogue, _transcriber, _csv_logger, _stt_logger

    log.info("■ Initializing singletons...")
    from rag.multimodal_rag import MultiModalRAG
    from audio.tts import VoiceEngine
    from ai.llm import Brain
    from pedagogy.dialogue import DialogManager
    from audio.transcriber import Transcriber
    from observability.logger import CSVMetricsLogger, STTLogger

    _rag = MultiModalRAG()
    _voice = VoiceEngine()
    _brain = Brain()
    _dialogue = DialogManager()
    _transcriber = Transcriber()
    _csv_logger = CSVMetricsLogger()
    _stt_logger = STTLogger()
    log.info("■ Singletons ready")

async def close_singletons():
    """Called at shutdown."""
    if _rag:
        _rag.save_caches()
        log.info("■ Singletons closed")

def get_rag(): return _rag
def get_voice(): return _voice
```

```
def get_brain(): return _brain
def get_dialogue(): return _dialogue
def get_transcriber(): return _transcriber
def get_csv_logger(): return _csv_logger
def get_stt_logger(): return _stt_logger
```

Tous les `from deps import get_rag` autour du codebase tirent ces singletons.

32. rag/metadata.py

Rôle : TypedDict pour le schema metadata Qdrant (catch typos).

```
from typing import TypedDict, Optional
```

```
class RAGDocumentMetadata(TypedDict, total=False):
    source_file: str
    chunk_idx: int
    idea_index_in_chunk: int
    domain: str
    course: str
    language: str
    original_text: str
    content_hash: str
    chapter_idx: int
    chapter_title: str
    section_idx: int
    section_title: str
    idea_id: str
    idea_label: str # definition|theorem|procedure|example|warning|note|fragment
    slide_idx: int
    image_url: str
    depends_on_ids: list[str]
    illustrates_id: Optional[str]
```

Type checker (mypy / pyright) flagge un `metadata["coursr"] = ...` (typo) → catch les bugs avant runtime.

33. rag/evaluation/

Sous-dossier dédié à l'évaluation du RAG (Recall@K, MRR, NDCG).

33.1 dataset.py

```
@dataclass
class RAGEvalQuery:
    query: str
    expected_chunk_ids: list[str] # ground truth (annotated)
    course_id: str
    metadata: dict

def load_dataset(path: str) -> list[RAGEvalQuery]:
    """Load JSONL eval dataset."""
    with open(path) as f:
        return [RAGEvalQuery(**json.loads(line)) for line in f]
```

33.2 metrics.py

```
def recall_at_k(retrieved_ids: list[str], expected_ids: list[str], k: int) -> float:
    if not expected_ids:
        return 0.0
    top_k = set(retrieved_ids[:k])
    hits = top_k & set(expected_ids)
    return len(hits) / len(expected_ids)

def mrr(retrieved_ids: list[str], expected_ids: list[str]) -> float:
    """Mean Reciprocal Rank : 1/rank of first relevant doc."""
    expected_set = set(expected_ids)
    for i, rid in enumerate(retrieved_ids, start=1):
        if rid in expected_set:
            return 1.0 / i
    return 0.0

def ndcg_at_k(retrieved_ids, expected_ids, k: int = 10) -> float:
    """Normalized Discounted Cumulative Gain."""
    expected_set = set(expected_ids)
    dcg = sum(1.0 / math.log2(i + 1) for i, rid in enumerate(retrieved_ids[:k], start=1)
              if rid in expected_set)
    ideal_dcg = sum(1.0 / math.log2(i + 1) for i in range(1, min(k, len(expected_ids)) + 1))
    return dcg / ideal_dcg if ideal_dcg > 0 else 0.0
```

33.3 runner.py

```
async def run_eval(dataset_path: str, rag, top_k: int = 5) -> dict:
    queries = load_dataset(dataset_path)
    metrics = {"recall@1": [], "recall@5": [], "mrr": [], "ndcg@10": []}

    for q in queries:
        chunks = await asyncio.to_thread(
            rag.retrieve_chunks, q.query, k=top_k, course_id=q.course_id
        )
        retrieved_ids = [c[0].metadata.get("idea_id") for c in (chunks or [])]
        metrics["recall@1"].append(recall_at_k(retrieved_ids, q.expected_chunk_ids, 1))
        metrics["recall@5"].append(recall_at_k(retrieved_ids, q.expected_chunk_ids, 5))
        metrics["mrr"].append(mrr(retrieved_ids, q.expected_chunk_ids))
        metrics["ndcg@10"].append(ndcg_at_k(retrieved_ids, q.expected_chunk_ids, 10))

    return {k: sum(v) / max(1, len(v)) for k, v in metrics.items()}
```

Utilisé par `scripts/run_rag_eval.py` pour reproducible quality benchmarking.

34. storage/transcript_search.py

Rôle : recherche full-text dans transcripts (Elasticsearch + fallback memory).

ES_INDEX = "smart_teacher_transcripts"

```
@dataclass
class TranscriptEntry:
    session_id: str
    language: str
    course_id: str
    course_title: str
    role: str # "student" | "teacher"
    text: str
```

```

subject:      str = ""
timestamp:    float = 0.0

def to_doc(self) -> dict:
    return {
        "session_id": self.session_id,
        "language": self.language,
        "course_id": self.course_id,
        "course_title": self.course_title,
        "role": self.role,
        "text": self.text,
        "subject": self.subject,
        "@timestamp": datetime.utcnow().isoformat(),
    }

class TranscriptSearch:
    def __init__(self):
        self._use_es = False
        self._memory_index: list[TranscriptEntry] = []
        self._session = None
        self._init_es()

    def _init_es(self):
        if not Config.ES_URL:
            log.info("■ Elasticsearch non configuré → recherche en mémoire")
            return
        try:
            self._session = requests.Session()
            r = self._session.get(self._es_url(""), timeout=2)
            r.raise_for_status()
            self._ensure_index()
            self._use_es = True
            cluster_info = r.json()
            log.info("■ Elasticsearch connecté : %s (cluster=%s, status=%s)",
                    Config.ES_URL, cluster_info.get("cluster_name"),
                    cluster_info.get("cluster_status"))
        except Exception as exc:
            log.info("■■ Elasticsearch non dispo (%s) → recherche en mémoire", exc)

    def index(self, entry: TranscriptEntry) -> bool:
        if not entry.timestamp:
            entry.timestamp = time.time()

        if self._use_es:
            try:
                t0 = time.time()
                resp = self._session.post(
                    self._es_url(f"{ES_INDEX}/_doc"),
                    headers=self._es_headers(),
                    timeout=2, auth=self._es_auth,
                    json=entry.to_doc(),
                )
                resp.raise_for_status()
                log.info("■ ES INDEX | index=%s role=%s lang=%s | text_len=%d | took=%.0fms",
                        ES_INDEX, entry.role, entry.language,
                        len(entry.text or ""), (time.time() - t0) * 1000)
                return True
            except Exception as exc:
                log.warning("■ ES INDEX FAIL (%s) → fallback memory", exc)
                self._use_es = False

        # Memory fallback
        self._memory_index.append(entry)
        if len(self._memory_index) > 5000:
            self._memory_index.pop(0)
        log.debug("■ MEMORY INDEX | role=%s lang=%s | total_in_mem=%d/5000",
                entry.role, entry.language, len(self._memory_index))
        return True

    def search(self, query: str, language="", course_id="", role="", limit=20) -> list[dict]:

```

```

if self._use_es:
    return self._es_search(query, language, course_id, role, limit)
return self._memory_search(query, language, course_id, role, limit)

def _es_search(self, query, language, course_id, role, limit) -> list[dict]:
    must = [{"multi_match": {
        "query": query,
        "fields": ["text^3", "course_title^2", "subject"],
        "type": "best_fields",
    }}]
    filters = []
    if language: filters.append({"term": {"language": language}})
    if course_id: filters.append({"term": {"course_id": course_id}})
    if role: filters.append({"term": {"role": role}})

    body = {
        "query": {"bool": {"must": must, "filter": filters}},
        "sort": [{"@timestamp": "desc"}],
        "size": limit,
        "highlight": {"fields": {"text": {}}},
    }
    try:
        t0 = time.time()
        resp = self._session.post(self._es_url(f"{ES_INDEX}/_search"),
                                   headers=self._es_headers(), timeout=2,
                                   auth=self._es_auth, json=body)

        resp.raise_for_status()
        r = resp.json()
        log.info("■ ES SEARCH DONE | hits=%d/%d | took_es=%dms total=%.0fms",
                 len(r.get("hits", {}).get("hits", [])),
                 r.get("hits", {}).get("total", {}).get("value", 0),
                 int(r.get("took") or 0),
                 (time.time() - t0) * 1000)
        return [hit["_source"] | {"highlight": hit.get("highlight", {})}
                for hit in r.get("hits", {}).get("hits", [])]
    except Exception:
        return self._memory_search(query, language, course_id, role, limit)

```

35. storage/media_storage.py

Rôle : abstraction MinIO / local filesystem pour les médias (PDF, slides PNG, audio MP3).

Voir Part 2 §28 pour overview. Logique principale :

```

class MediaStorage:
    def __init__(self):
        self._minio = None
        self._use_minio = False
        self._init_storage()

    def _init_storage(self):
        if not Config.MINIO_ENDPOINT:
            log.info("■ Using local storage (MinIO not configured)")
            return
        try:
            from minio import Minio
            self._minio = Minio(
                Config.MINIO_ENDPOINT,
                access_key=Config.MINIO_ACCESS_KEY,
                secret_key=Config.MINIO_SECRET_KEY,
                secure=Config.MINIO_SECURE,
            )
            if not self._minio.bucket_exists(Config.MINIO_BUCKET):
                self._minio.make_bucket(Config.MINIO_BUCKET)

```

```

        log.info("■ MinIO bucket created: %s", Config.MINIO_BUCKET)
    else:
        log.info("■ MinIO connected: %s/%s", Config.MINIO_ENDPOINT, Config.MINIO_BUCKET)
        self._use_minio = True
    except Exception as exc:
        log.warning("■ MinIO unavailable (%s) → local storage fallback", exc)

def upload_bytes(self, data: bytes, object_name: str, content_type: str) -> str:
    t0 = time.time()
    size_kb = len(data) / 1024.0
    backend = "minio" if self._use_minio else "local"
    log.info("■ STORAGE upload START | backend=%s object=%s size=%dB (%.1fKB) mime=%s",
            backend, object_name, len(data), size_kb, content_type)
    if self._use_minio:
        url = self._minio_upload_bytes(data, object_name, content_type)
    else:
        url = self._local_save_bytes(data, object_name)
    elapsed = time.time() - t0
    throughput = (size_kb / elapsed) if elapsed > 0 else 0
    log.info("■ STORAGE upload DONE | backend=%s | took=%.0fms throughput=%.0fKB/s | url=%s",
            backend, elapsed * 1000, throughput, url[:80])
    return url

def get_url(self, object_name: str, expires_hours: int = 24) -> str:
    if self._use_minio:
        from datetime import timedelta
        return self._minio.presigned_get_object(
            Config.MINIO_BUCKET, object_name,
            expires=timedelta(hours=expires_hours))
    return f"/media/{object_name}"

def get_bytes(self, object_name: str) -> Optional[bytes]:
    if self._use_minio:
        try:
            response = self._minio.get_object(Config.MINIO_BUCKET, object_name)
            data = response.read()
            response.close()
            return data
        except Exception:
            return None
    path = LOCAL_MEDIA_DIR / object_name
    return path.read_bytes() if path.exists() else None

# Helpers métier
def save_course_pdf(self, course_id: str, filename: str, data: bytes) -> str:
    return self.upload_bytes(data, f"pdfs/{course_id}/{filename}", "application/pdf")

def save_audio(self, session_id: str, data: bytes, mime: str = "audio/mpeg") -> str:
    ts = int(time.time())
    ext = "mp3" if "mpeg" in mime else "webm"
    return self.upload_bytes(data, f"audio/{session_id}/{ts}.{ext}", mime)

def save_slide_image(self, course_id: str, slide_idx: int, data: bytes, **kwargs) -> str:
    if all(k in kwargs for k in ("domain", "course", "chapter")):
        obj = f"slides/{kwargs['domain']}/{kwargs['course']}/{kwargs['chapter']}/page_{slide_idx:03d}.png"
    else:
        obj = f"slides/{course_id}/slide_{slide_idx:04d}.png"
    return self.upload_bytes(data, obj, "image/png")

```

36. observability/dashboard.py

Rôle : route HTML pour dashboard ops/teachers.

```

@router.get("/dashboard", response_class=HTMLResponse)
async def dashboard():
    """Render Grafana-like dashboard."""
    services = await get_services_status()
    rag_stats = deps.get_rag().get_stats()
    llm_stats = get_default_router().stats.snapshot()

    html = f"""<!DOCTYPE html>
<html><head><title>Smart Teacher Dashboard</title>
<link rel="stylesheet" href="/static/dashboard.css"></head>
<body>
<h1>Smart Teacher Dashboard</h1>
<section class="services">
    <h2>Services</h2>
    <ul>
        {"".join(f'<li><span class="{v}">{k}: {v}</span></li>'
                  for k, v in services["services"].items())}
    </ul>
</section>
<section class="rag">
    <h2>RAG</h2>
    <p>Docs: {rag_stats.get('total_docs', 0)} | Embed dim: {rag_stats.get('embedding_dim')}</p>
</section>
<section class="llm">
    <h2>LLM Stats</h2>
    <pre>{json.dumps(llm_stats, indent=2)}</pre>
</section>
</body></html>
"""
    return HTMLResponse(content=html)

```

Beaucoup plus simple que Grafana en pratique, mais suffisant pour debug rapide.

37. database/crud.py

Rôle : helpers CRUD courants.

```

async def create_learning_session(session_id, student_id, course_id, language) -> uuid.UUID:
    async with AsyncSessionLocal() as db:
        ls = LearningSession(
            id=uuid.uuid4(),
            student_id=_coerce_uuid(student_id),
            course_id=_coerce_uuid(course_id),
            session_id_str=str(session_id),
            language=language,
            started_at=datetime.utcnow(),
        )
        db.add(ls)
        await db.commit()
        await db.refresh(ls)
        log.info("Created learning session: %s for student %s", ls.id, session_id[:8])
        return ls.id

async def get_all_courses(db) -> list[Course]:
    stmt = select(Course).order_by(Course.created_at.desc())
    return (await db.execute(stmt)).scalars().all()

async def insert_interaction(student_id, course_id, session_id, question, answer, lang, subject):
    async with AsyncSessionLocal() as db:
        i = Interaction(
            id=uuid.uuid4(),
            student_id=_coerce_uuid(student_id),

```

```

        course_id=_coerce_uuid(course_id),
        session_id_str=session_id,
        question=question[:5000],
        answer=answer[:5000],
        language=lang,
        subject=subject,
        timestamp=datetime.utcnow(),
    )
    db.add(i)
    await db.commit()

```

38. database/reset_db_fresh.py

Rôle : nuclear option DROP + CREATE all tables. À utiliser avec PRUDENCE.

```

async def reset_database_fresh():
    """■■■ DROP all tables and recreate them. ALL DATA LOST."""
    async with engine.begin() as conn:
        # Drop schema cascade
        await conn.execute(text("DROP SCHEMA IF EXISTS public CASCADE"))
        await conn.execute(text("CREATE SCHEMA public"))
        await conn.execute(text("GRANT ALL ON SCHEMA public TO admin"))
        # Recreate tables
        await conn.run_sync(Base.metadata.create_all)
    log.warning("■■■ Database reset FRESH – all data lost")
    return True

if __name__ == "__main__":
    answer = input("■■■ DROP all tables ? [yes/N] ").strip()
    if answer.lower() == "yes":
        asyncio.run(reset_database_fresh())

```

39. scripts/reset_for_reingest.py

Rôle : reset orchestré pour re-ingestion d'un cours sans tout casser.

```

python scripts/reset_for_reingest.py [--yes] [--keep-titles] [--keep-disk]
                                     [--keep-qdrant] [--keep-redis] [--keep-db]

```

Cf. Part 1 §6 Auth section (déjà détaillé). Étapes : 1. Disk caches : `cache/vision_descriptions/`, `data/kg_concepts_cache.json`, `data/multimodal_db/{docs,idea,summary}_cache.json` 2. Slide-title LLM cache (séparable via `--keep-titles`) 3. Qdrant collections matching `smart_teacher_multimodal*` 4. Redis : `qa_cache:*`, `slide_cache:*`, `embedding_cache:*`, `emb:*` 5. Postgres tables : `rag_chunks`, `concepts`, `ingested_asset`, `sections`, `chapters`, `courses`

Étudiants + mastery + bandit + FSRs sont **préservés** (utilise `database/reset_db_fresh.py` pour tout virer).

40. scripts/reset_students_and_cache.py

Rôle : variante qui vide les étudiants + caches mais garde les cours.

Use case : faire des tests login/onboarding sans re-ingérer le PDF.

```
python scripts/reset_students_and_cache.py [--apply]
```

Tables truncated :

- `students, learning_sessions, interactions, learning_events`
- `student_profiles, student_mistakes, student_mastery`
- `practice_attempt, review_queue, learning_gain_tests`
- `confusion_events, vark_responses`

Redis keys cleared :

- `chat::*, presentation:snapshot:*, narration:*`
- `session:*, seen_ideas:*, session_token:*`

Cours, slides, RAG → **préservés**.

41. scripts/bootstrap_bandit.py

Rôle : initialise les arms du bandit avec prior Beta(1, 1).

```
async def bootstrap_bandit_for_course(course_id: str):
    """Insert all (bucket × arm) combinations with Beta(1, 1) prior."""
    BUCKETS = [
        f"{conf}|{eng}|{mast}|{intr}"
        for conf in ("low", "medium", "high")
        for eng in ("disengaged", "neutral", "engaged")
        for mast in ("low", "medium", "high")
        for intr in ("isolated", "mixed", "dense")
    ]
    STRATEGIES = ["socratic", "simpler_words", "decomposition", "analogy", "recap", "example"]
    SPEEDS = ["slow", "normal", "fast"]
    ARMS = [f"{s}:{r}" for s in STRATEGIES for r in SPEEDS]

    from pedagogy.personalization.bandit.repo import BanditRepo
    repo = BanditRepo()
    n = 0
    for bucket in BUCKETS:
        for arm in ARMS:
            await repo.set_posterior(course_id, bucket, arm, alpha=1.0, beta=1.0, n_pulls=0)
            n += 1
    log.info("Bootstrapped %d (bucket, arm) pairs for course %s", n, course_id[:8])
```

À lancer après l'ingestion d'un nouveau cours (optionnel — les arms se créent à la volée à la première utilisation).

42. scripts/extract_bandit_logs.py

```
python scripts/extract_bandit_logs.py --start "2026-05-01" --end "2026-05-04" --output samples.json
```

Parse les logs serveur → JSON dataset (utilisable par `pedagogy.personalization.bandit.offline.rebuild_posteriors_from_logs`).

43. scripts/profile_agentic.py

```
python scripts/profile_agentic.py --turns 100 --course-id 9940c8f4
```

Lance le Q&A graph 100 fois sur des questions synthétiques, sort le breakdown timing par node.

Output typique :

```
== Q&A graph profiling (100 runs) ==
intent      | mean=0.62s p50=0.55s p95=0.92s
rewriter    | mean=0.00s p50=0.00s p95=0.00s   (skipped 100% of time)
retriever   | mean=1.45s p50=1.32s p95=2.10s
responder   | mean=1.80s p50=1.65s p95=2.60s
qa_review   | mean=0.45s p50=0.40s p95=0.65s
TOTAL       | mean=4.32s p50=3.92s p95=6.27s
```

44. scripts/run_rag_eval.py

```
python scripts/run_rag_eval.py --dataset rag/evaluation/eval_set.jsonl
```

Lance le RAG sur un dataset annoté, sort metrics Recall@K, MRR, NDCG.

45. scripts/clear_presentation_cache.py

```
python scripts/clear_presentation_cache.py [--session-id SID] [--all]
```

Clear sélectif :

- Sans args : help message
- `--session-id` : juste les snapshots de cette session
- `--all` : toutes les snapshots (tous les students)

```
async def clear_session_snapshots(session_id: str):
    from pedagogy.dialogue import get_redis
    r = await get_redis()
    pattern = f"presentation:snapshot:{session_id}*"
    keys = list(r.scan_iter(match=pattern, count=500))
    if keys:
        await r.delete(*keys)
    log.info("Cleared %d snapshots for session %s", len(keys), session_id[:8])
```

46. scripts/e2e_ingestion_test.py

Rôle : test end-to-end ingest (PDF → DB → RAG → KG → Q&A retrieval) pour CI.

```
async def main():
    # 1. Ingest a fixture PDF
```

```

fixture = "tests/fixtures/sample_course.pdf"
builder = CourseBuilder()
course_data = await builder.build_from_file(fixture, language="fr", level="lycée")

# 2. Save to DB
async with AsyncSessionLocal() as db:
    course_id = await builder.save_to_database(course_data, db, domain="general")

# 3. Index to RAG
rag = MultiModalRAG()
rag_ok = rag.run_ingestion_pipeline_from_course_data(course_data, course_id=course_id)
assert rag_ok, "RAG ingestion failed"

# 4. Verify retrieval
chunks = rag.retrieve_chunks("what is the main concept?", k=5, course_id=course_id)
assert len(chunks) > 0, "No chunks retrieved"

# 5. Verify KG
kg = get_or_build(rag)
assert len(kg) > 0, "KG empty"

print("■ E2E ingestion test passed")

```

47. scripts/migrate_phase2_persistence.py

Rôle : migration one-shot pour bandit Phase 2 (de Redis → Postgres).

```

async def migrate_bandit_redis_to_postgres():
    """Phase 1 stored bandit state in Redis. Phase 2 moves to Postgres
    for durability + queryability."""
    r = await get_redis()
    n_migrated = 0
    cursor = 0
    while True:
        cursor, keys = await r.scan(cursor=cursor, match="bandit:posterior:*", count=500)
        for key in keys:
            data = json.loads(await r.get(key))
            # Parse key : "bandit:posterior:{course_id}:{bucket}:{arm}"
            _, _, course_id, bucket, arm = key.split(":")
            await BanditRepo().set_posterior(course_id, bucket, arm,
                                             alpha=data["alpha"], beta=data["beta"],
                                             n_pulls=data.get("n_pulls", 0))

            n_migrated += 1
        if cursor == 0:
            break
    log.info("Migrated %d Beta posteriors Redis → Postgres", n_migrated)

```

48. analyze_metrics.py

Rôle : script standalone pour analyser le CSV des metrics et imprimer un rapport.

```

import pandas as pd
from datetime import datetime, timedelta

def main():
    df = pd.read_csv(Config.CSV_LOG_FILE)
    df["timestamp"] = pd.to_datetime(df["timestamp"])

    # Last 24h
    last_24h = df[df["timestamp"] > datetime.utcnow() - timedelta(hours=24)]

```

```

print(f"■ Smart Teacher Metrics – Last 24h")
print(f"  Total turns:      {len(last_24h)}")
print(f"  Avg STT time:       {last_24h.stt_time.mean():.2f}s")
print(f"  Avg LLM time:        {last_24h.llm_time.mean():.2f}s")
print(f"  Avg TTS time:        {last_24h.tts_time.mean():.2f}s")
print(f"  Avg total:           {last_24h.total_time.mean():.2f}s")
print(f"  KPI pass rate:       {last_24h.kpi_ok.mean():.1%}")
print(f"  By language:         {last_24h.language.value_counts().to_dict()}")
print()
print(f"■ Latency distribution (total_time)")
print(f"  p50:                 {last_24h.total_time.quantile(0.50):.2f}s")
print(f"  p95:                 {last_24h.total_time.quantile(0.95):.2f}s")
print(f"  p99:                 {last_24h.total_time.quantile(0.99):.2f}s")
print(f"  max:                 {last_24h.total_time.max():.2f}s")

```

Fin de Part 3.

*Total : 48 fichiers documentés en Part 3, 77 en Parts 1+2, soit **125 fichiers couverts** sur 166. Les 41 restants sont :*

- *Tests (~30 fichiers — code self-documenting via assert names)*
- *`__init__.py` (re-exports)*
- *`audio/voice/__init__.py`, `pedagogy/__init__.py`, etc.*
- *Notebooks / scripts d'expérimentation*
- *`pedagogy/intelligent_ingester.py` extracteurs internes (helpers privés)*

Pour le code restant, voir directement les fichiers source.